Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Гаврилюк Г.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.09.26

Москва, 2026

Постановка задачи

Вариант 8.

Родительский процесс создаёт дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида «число число число<endline>». Дочерний процесс производит деление первого числа команды на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип int, количество чисел в строке произвольное.

Общий метод и алгоритм решения

Использованные системные вызовы:

- ssize_t read(int fd, void *buf, size_t count); – читает до count байт из дескриптора fd в буфер buf, возвращает число прочитанных байт, 0 при конце данных или -1 при ошибке.
- ssize_t write(int fd, const void *buf, size_t count); – записывает count байт из буфера buf в дескриптор fd.
- int open(const char *pathname, int flags); – открывает файл и возвращает его дескриптор (здесь с флагом O_RDONLY, только чтение).
- int close(int fd); – закрывает дескриптор.
- int pipe(int pipefd[2]); – создаёт однонаправленный канал: pipefd[0] – конец для чтения, pipefd[1] – конец для записи.
- pid_t fork(void); – создаёт дочерний процесс, копию текущего. В ребёнке возвращает 0, в родителе – PID ребёнка.
- int dup2(int oldfd, int newfd); – делает так, что дескриптор newfd ссылается на тот же объект, что и oldfd. Используется для подмены stdin и stdout ребёнка.
- int execv(const char *path, char *const argv[]); – заменяет программу текущего процесса на программу из файла path. При успехе не возвращает управление.
- pid_t waitpid(pid_t pid, int *status, int options); – ждёт завершения указанного дочернего процесса и сохраняет в status информацию о том, как он завершился.
- void _exit(int status); – немедленно завершает процесс с заданным кодом.

Работа разбита на две программы: parent.cpp (родитель) и child.cpp (ребёнок), каждая компилируется в отдельный исполняемый файл. Стандартные библиотеки ввода-вывода не используются, весь ввод и вывод идёт через read и write, а числа переводятся из строки в int и обратно вручную.

Родитель. Сначала посимвольно читает из stdin имя файла до перевода строки и открывает этот файл на чтение. Затем создаёт канал pipe1 и вызывает fork. В ветке ребёнка через dup2 подменяет stdin открытым файлом, а stdout – концом канала для записи, закрывает лишние дескрипторы и через execv запускает программу child. Сама программа child про файл и канал ничего не знает: она просто

читает свой stdin и пишет в свой stdout. В ветке родителя закрываются файл и конец канала для записи (иначе read никогда не вернёт 0 и родитель зависнет), после чего родитель в цикле читает данные из канала блоками по 4096 байт и выводит их в свой stdout, пока канал не закроется. В конце он дожидается ребёнка через waitpid и, если тот завершился с ненулевым кодом, тоже завершается с ошибкой.

Ребёнок. Читает stdin блоками по 4096 байт и собирает из них строки: символы копятся в накопителе до перевода строки, поэтому строка, разрезанная между двумя вызовами read, склеивается правильно. Каждая строка разбирается на числа: пропускаются пробелы, учитывается знак минус, цифры собираются в число, проверяется выход за пределы int. Пустые строки пропускаются. Затем первое число по очереди делится на все остальные; перед каждым делением проверяется делитель, и при нуле ребёнок пишет сообщение в stderr и завершается с кодом 1. Результат переводится в строку (цифры выкладываются в массив с конца) и записывается в stdout, то есть в канал.

Все системные вызовы проверяются на ошибку; при ошибке сообщение выводится в stderr и процесс завершается с кодом 1. Для сборки и прогона тестов написан Makefile (make, make test, make clean).

Код программы

parent.cpp

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string>
using namespace std;

void print_error(const string& error) {
    write(STDERR_FILENO, error.c_str(), error.size());
    _exit(1);
}

int main() {
    string name;

    while (true) { // read and save name of file
        char c;
        ssize_t n = read(STDIN_FILENO, &c, 1);

        if (n == -1) {
            print_error("error: failed to read file name\n");
        }
        if (n == 0) {
            break;
        }
        if (c == '\n') {
            break;
        }
        name += c;
    }
    if (name.empty()) {
        print_error("error: file name is empty\n");
    }

    int opened_file = open(name.c_str(), O_RDONLY); // open file
```

```
if (opened_file == -1) {
    print_error("error: failed to open file\n");
}

int pipe1[2]; // create pipe (pipe for conversation child -> parent)
if (pipe(pipe1) == -1) {
    print_error("error: failed to create pipe\n");
}

pid_t pid = fork();

if (pid == -1) {
    print_error("error: failed to fork\n");
}
else if (pid == 0) {
    close(pipe1[0]);

    if (dup2(opened_file, STDIN_FILENO) == -1) {
        print_error("error: failed to redirect stdin\n");
    }
    if (dup2(pipe1[1], STDOUT_FILENO) == -1) {
        print_error("error: failed to redirect stdout\n");
    }

    close(opened_file);
    close(pipe1[1]);

    char* args[] = { (char*)"child", nullptr };
    execv("./child", args);
    print_error("error: failed to exec child\n");
}
else {
    close(opened_file);
    close(pipe1[1]);

    char buf[4096];
    while (true) {
        ssize_t n = read(pipe1[0], buf, sizeof(buf));

        if (n == -1) {
            print_error("error: failed to read from pipe\n");
        }
        if (n == 0) {
            break;
        }

        if (write(STDOUT_FILENO, buf, n) == -1) {
            print_error("error: failed to write to stdout\n");
        }
    }
    close(pipe1[0]);

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        print_error("error: failed to wait for child\n");
    }

    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        print_error("error: child process failed\n");
    }
}
```

```
    return 0;
}
```

child.cpp

```
#include <unistd.h>
#include <string>
#include <vector>
using namespace std;

void print_error(const string& error) {
    write(STDERR_FILENO, error.c_str(), error.size());
    _exit(1);
}

void process_line(const string& line) {
    vector<int> nums;
    size_t i = 0;

    while (i < line.size()) {
        while (i < line.size() && (line[i] == ' ' || line[i] == '\t' || line[i] == '\r')) {
            i++;
        }

        if (i == line.size()) {
            break;
        }

        bool negative = false;
        if (line[i] == '-') {
            negative = true;
            i++;
        }

        if (i == line.size() || line[i] < '0' || line[i] > '9') {
            print_error("error: invalid number\n");
        }

        long long value = 0;
        while (i < line.size() && line[i] >= '0' && line[i] <= '9') {
            value = value * 10 + (line[i] - '0');
            if (value > 2147483648) {
                print_error("error: number out of range\n");
            }
            i++;
        }

        if (negative) {
            value = -value;
        }
        if (value > 2147483647) {
            print_error("error: number out of range\n");
        }

        nums.push_back((int)value);
    }

    if (nums.empty()) {
        return;
    }
```

```
    int result = nums[0];

    for (size_t j = 1; j < nums.size(); j++) {
        if (nums[j] == 0) {
            print_error("error: division by zero\n");
        }
        result = result / nums[j];
    }

    char digits[16];
    int pos = 16;

    digits[--pos] = '\n';

    long long x = result;
    bool negative = x < 0;
    if (negative) {
        x = -x;
    }

    do {
        digits[--pos] = (char)('0' + x % 10);
        x = x / 10;
    } while (x > 0);

    if (negative) {
        digits[--pos] = '-';
    }

    if (write(STDOUT_FILENO, digits + pos, 16 - pos) == -1) {
        print_error("error: failed to write result\n");
    }
}

int main() {
    char buf[4096];
    string line;

    while (true) {
        ssize_t n = read(STDIN_FILENO, buf, sizeof(buf));

        if (n == -1) {
            print_error("error: failed to read input\n");
        }
        if (n == 0) {
            break;
        }

        for (ssize_t i = 0; i < n; i++) {
            if (buf[i] == '\n') {
                process_line(line);
                line.clear();
            } else {
                line += buf[i];
            }
        }
    }

    if (!line.empty()) {
```

```
        process_line(line);
    }

    return 0;
}
```

Протокол работы программы

Тестирование:

```
$ make
g++ -Wall -Wextra -o parent parent.cpp
g++ -Wall -Wextra -o child child.cpp

$ cat ok.txt
100 5 2
81 3 3
-20 4
$ ./parent
ok.txt
10
9
-5
$ echo $?
0


$ cat edge.txt
7 2
-7 2
100 -5 2
0 5
42
$ ./parent
edge.txt
3
-3
-10
0
42


$ cat zero.txt
100 5 2
81 3 3
-20 4
7 0 1
50 2
$ ./parent
zero.txt
10
9
-5
error: division by zero
error: child process failed
$ echo $?
1


$ cat bad.txt
100 abc
$ ./parent
bad.txt
error: invalid number
error: child process failed
```

```
$ ./parent
nofile.txt
error: failed to open file

$ ./parent

error: file name is empty

$ ./child < ok.txt
10
9
-5
```

Вывод

В ходе работы я разобрался, как в Linux устроены процессы и их взаимодействие на уровне системных вызовов: как fork создаёт копию процесса, как execv заменяет в ней программу, как через pipe передавать данные между процессами и как через dup2 подменять стандартные потоки, чтобы дочерняя программа работала с файлом и каналом, ничего о них не зная. Получилось две программы, которые делают то же, что оболочка делает при команде вида «./child < file | ...». Основные сложности были с базовыми понятиями (процесс, дескриптор, канал) и с тем, что без stdio пришлось вручную разбирать числа из строки и переводить результат обратно в текст. Также важно было не забыть закрыть неиспользуемые концы канала, иначе родитель зависает в ожидании данных.